

AIM:

To write a C# program to implement the concept of classes and objects.

ALGORITHM:**Program.cs**

- Step1. Start the program.
- Step2. Create a class named program
- Step3. Construct a main method.
- Step4. Create an object for the class studentdetail .
- Step5. Call the methods getdetail(), putdetail(), result(), position() using the object.
- Step6. End the class.
- Step7. Stop.

StudentDetail.cs

- Step1. Start the program
- Step2. Create a class named StudentDetail
- Step3. Construct a method named getdetail()
 - Step3.1. Get the name, roll no., marks (say m1,m2,m3)
 - Step3.2. Calculate the total and Average as
 - Total=m1+m2+m3
 - Average=Total/3
- Step4. Construct a method named putdetail()
 - Step4.1. Display the name, roll no. marks, total and average.
- Step5. Construct a method named Result()
 - Step5.1. if m1, m2, m3 are greater than 50
 - Print "pass"
 - Else Print "Fail"
- Step6. Construct another method named position()
 - Step6.1. if avg>=75 && avg<=100
 - Print Grade= Distinction.
 - Step6.2. if avg>=60 && avg<75
 - Print Grade= First class.
 - Step 6.3. if avg >= 50 && avg < 60
 - Print Grade=Second class.
 - Step 6.4. else
 - Print Grade=NIL.
- Step7. End the class
- Step8. End the program.

SOURCE CODE:

Program.cs [Main Program]

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
namespace Test
{
    class Program
    {
        static void Main(string[] args)
        {
            First Program
            StudentDetail sd = new StudentDetail();
            sd.getDetail();
            sd.putDetail();
            sd.Result();
            sd.Position();
            Console.Read();
        }
    }
}
```

StudentDetails.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
namespace Test
{
    public class StudentDetail
    {
        string name, enrollno;
        int m1, m2, m3, total;
        double avg;
        public void getDetail()
        {
            Console.WriteLine("Enter The Name:");
            name = Console.ReadLine();
            Console.WriteLine("Enter The EnrollNo:");
            enrollno=Console.ReadLine();
            Console.WriteLine("Enter Mark1:");
```

```
m1 = int.Parse(Console.ReadLine());
Console.WriteLine("Enter Mark12:");
m2 = int.Parse(Console.ReadLine());
Console.WriteLine("Enter Mark3:");
m3 = int.Parse(Console.ReadLine());
total = m1 + m2 + m3;
avg = Convert.ToDouble(total / 3.0);
}
```

```
public void putDetail()
{
    Console.WriteLine("Name:" + name);
    Console.WriteLine("EnrollNo:" + enrollno);
    Console.WriteLine("Mark1:" + m1);
    Console.WriteLine("Mark2:" + m2);
    Console.WriteLine("Mark3:" + m3);
    Console.WriteLine("Total:" + total);
    Console.WriteLine("Average:" + total/3.0);
}
```

```
public void Result()
{
    if (m1 >= 50 && m2 >= 50 && m3 >= 50)
    {
        Console.WriteLine("Result : PASS");
    }
    else
    {
        Console.WriteLine("Result : Fail");
    }
}
```

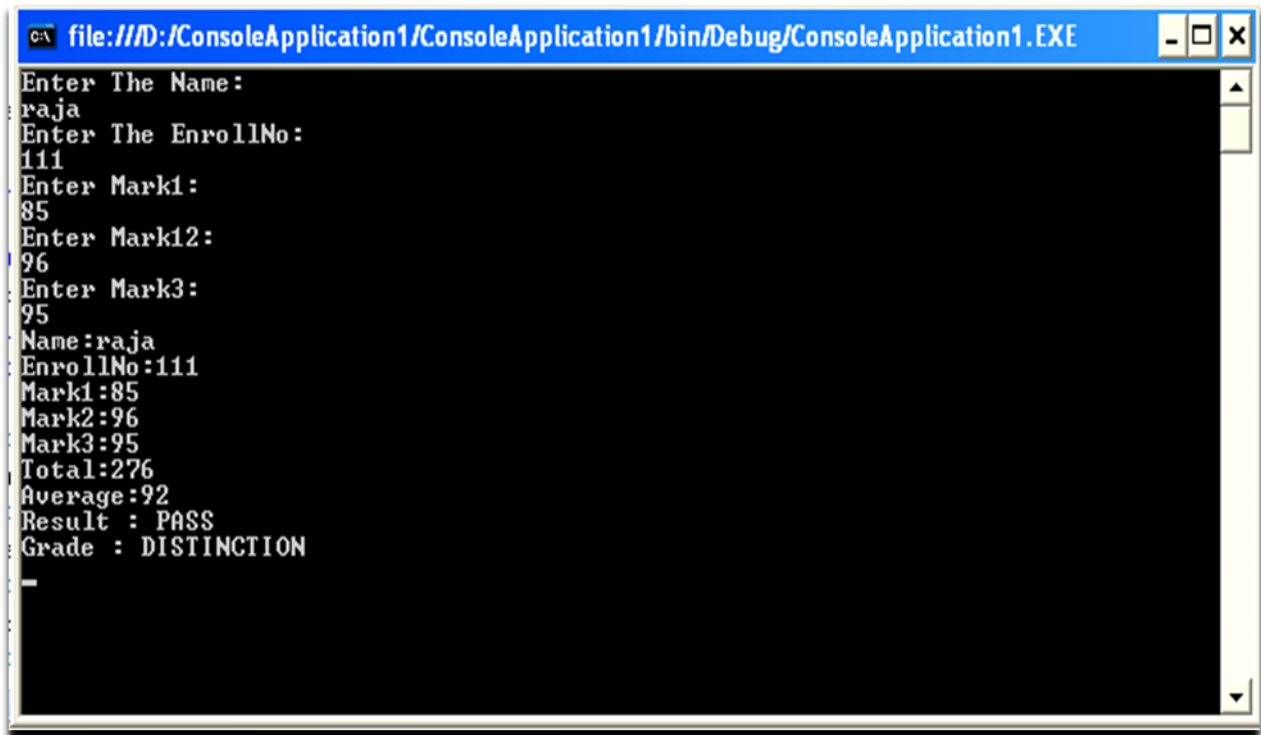
```
public void Position()
{
    if (avg >= 75 && avg <= 100)
    {
        Console.WriteLine("Grade : DISTINCTION");
    }
    else if (avg >= 60 && avg < 75)
    {
        Console.WriteLine("Grade : FIRST CLASS");
    }
    else if (avg >= 50 && avg < 60)
    {

```

```
        Console.WriteLine("Grade : SECOND CLASS");
    }

else
    {
        Console.WriteLine("Grade : NIL");
    }
}
}
```

OUTPUT:



```
file:///D:/ConsoleApplication1/ConsoleApplication1/bin/Debug/ConsoleApplication1.EXE
Enter The Name :
raja
Enter The EnrollNo :
111
Enter Mark1 :
85
Enter Mark2 :
96
Enter Mark3 :
95
Name : raja
EnrollNo : 111
Mark1 : 85
Mark2 : 96
Mark3 : 95
Total : 276
Average : 92
Result : PASS
Grade : DISTINCTION
```

Result:

Thus the C# program using classes and objects has been executed and the output is verified successfully.

AIM:

To write a C# program for implement inheritance.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a base class book with a getdata() method.

Step 3: Create a class publication derived from class book with the method getdata1().

Step 4: Create a class tape derived from class publication with the method getdata2().

Step 5: In getdata() method, get the bookname of the book.

Step 6: In getdata1() method, get the no of pages in a book and price of thebook

Step 7: In getdata2() method, get the reading reading time per page and display the reading time for book.

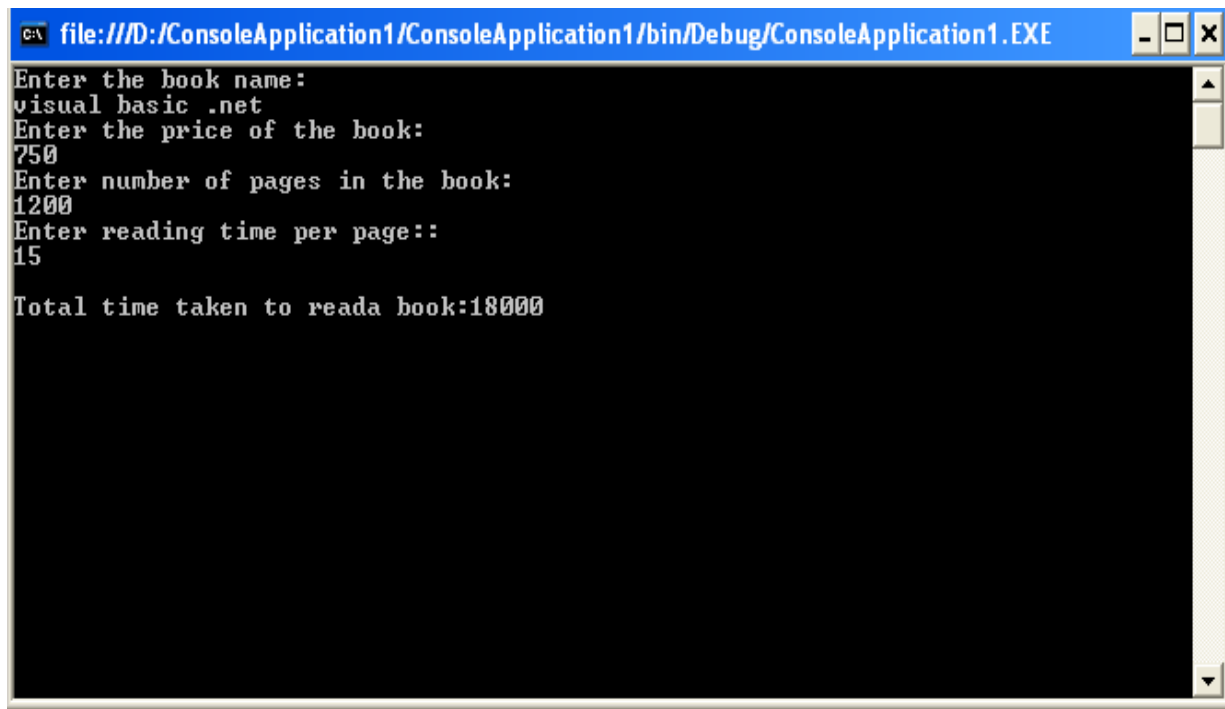
Step 8: create an object for a class tape, using an object call the getdata2() method to display the output.

Step 9: Stop the program.

SOURCE CODE:

```
using System;
using System.Collections.Generic;
using System.Text;
namespace Multilevel
{
    class book
    {
        public string name;
        public void getdata()
        {
            Console.WriteLine("Enter the book name:");
            name = Console.ReadLine();
        }
    }
    class publication:book
    {
        public int nop;
        public double price;
        public void getdata1()
        {
            getdata();
            Console.WriteLine("Enter the price of the book:");
            price = Convert.ToInt32(Console.ReadLine());
            Console.WriteLine("Enter number of pages in the book:");
            nop = Convert.ToInt32(Console.ReadLine());
        }
    }
    class tape:publication
    {
        public int time, total;
        public void getdata2()
        {
            getdata1();
```

```
Console.WriteLine("Enter reading time per page::");
time = Convert.ToInt32(Console.ReadLine());
total = nop * time;
Console.WriteLine("\nTotal time taken to read a book:" + total);
}
public static void Main(string[] args)
{
    tape obj = new tape();
    obj.getdata2();
    Console.ReadLine();
}
}
}
```


OUTPUT:

A screenshot of a Windows console application window. The title bar is blue and contains the text "file:///D:/ConsoleApplication1/ConsoleApplication1/bin/Debug/ConsoleApplication1.EXE" along with standard window control buttons (minimize, maximize, close). The console area has a black background with white text. The text shows a series of prompts and user inputs: "Enter the book name:", "visual basic .net", "Enter the price of the book:", "750", "Enter number of pages in the book:", "1200", "Enter reading time per page::", "15", and finally "Total time taken to read a book:18000".

```
file:///D:/ConsoleApplication1/ConsoleApplication1/bin/Debug/ConsoleApplication1.EXE
Enter the book name:
visual basic .net
Enter the price of the book:
750
Enter number of pages in the book:
1200
Enter reading time per page::
15
Total time taken to read a book:18000
```

RESULT:

Thus the program in C# to perform inheritance concept has been executed and the output is verified successfully.

AIM:

To write a C# program in order to display number of days in a month using function overloading in polymorphism.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a class daycount with a function as same as class name.

Step 3: Declare month and year as zero using constructors.

Step 4: Using leapyear function, find the given year is leapyear or not.

Step 5: Using the switch case, give separate cases for each month.

Step 6: In display function, display the number of days in the concerned month.

Step 7: Stop the program.

SOURCE CODE:

```
using System;
using System.Collections.Generic;
using System.Text;
namespace polymorphism
{
    class daycount
    {
        int month;
        int year;
        int days;
        public daycount()
        {
            month = 0;
            year = 0;
        }
        public Boolean leapyear()
        {
            if(year%4==0)
                return true;
            else
                return false;
        }
        public void setdate(int month1,int year1)
        {
            year = year1;
            month = month1;
        }
        public void setdate(int month1)
        {
            month=month1;
            year = 2006;
        }
    }
}
```

```
public string monthname()
{
    switch(month)
    {
        case 1: return "january";
        case 2: return "february";
        case 3: return "march";
        case 4: return "april";
        case 5: return "may";
        case 6: return "june";
        case 7: return "july";
        case 8: return "august";
        case 9: return "september";
        case 10: return "october";
        case 11: return "november";
        case 12: return "december";
        default: return "invalid month specified";
    }
}

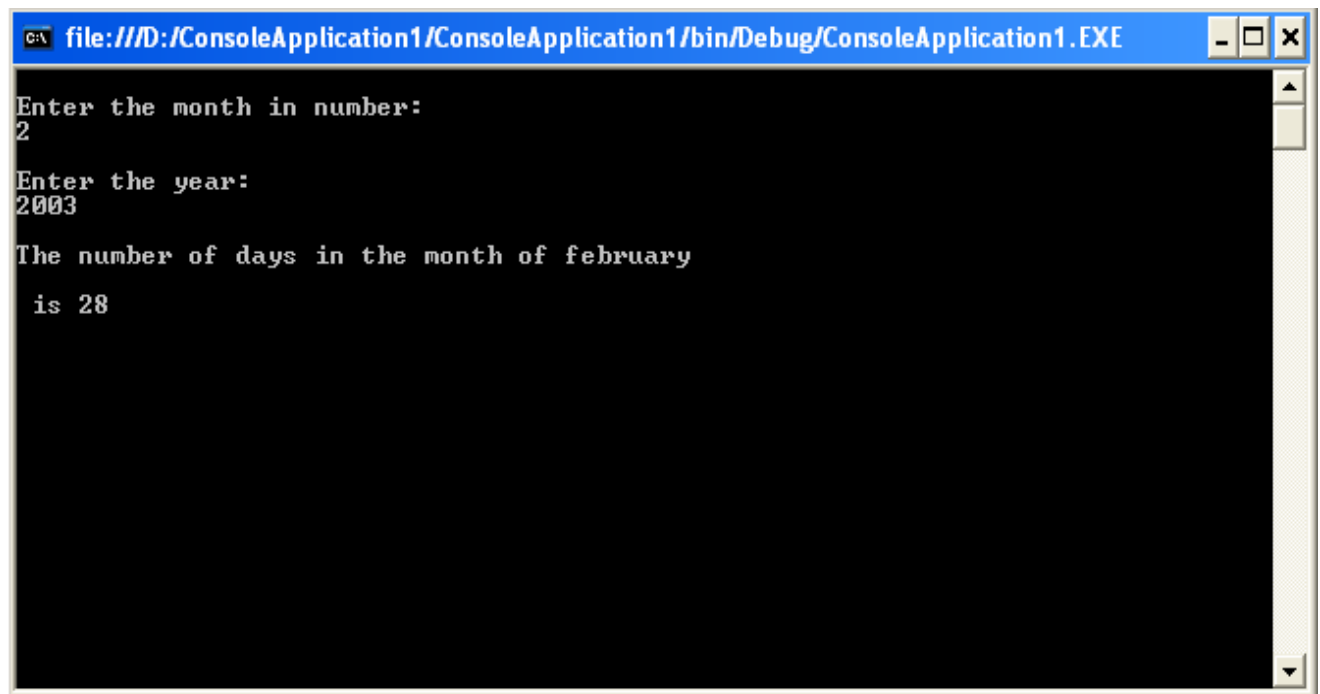
public void setdays()
{
    switch(month)
    {
        case 1:
        case 3:
        case 5:
        case 7:
        case 8:
        case 10:
        case 12:
            days = 31;
            break;
        case 2:
```

```

{
days = 28;
if (leapyear())
days = 29;0
break;
}
case 4:
case 6:
case 9:
case 11:
days = 30;
break;
}
}
public void display()
{
string[] name = new string[25];
setdays();
Console.WriteLine("\nThe number of days in the month of " + monthname());
Console.WriteLine("\n is " + days);
}
public static void Main(string[] args)
{
daycount dad=new daycount();
int month,year;
Console.WriteLine("\nEnter the month in number:");
month=Convert.ToInt32(Console.ReadLine());
Console.WriteLine("\nEnter the year:");
year=Convert.ToInt32(Console.ReadLine());
dad.setdate(month,year);
dad.display();
Console.ReadLine();
}
}
}

```

OUTPUT:



```
file:///D:/ConsoleApplication1/ConsoleApplication1/bin/Debug/ConsoleApplication1.EXE

Enter the month in number:
2

Enter the year:
2003

The number of days in the month of february
is 28
```

RESULT:

Thus the C# program using function overloading has been executed and the output is verified successfully.

AIM:

To write a program in C# for implement interface.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a class execute with a get1, get2 and display method.

Step 3: Declare the interface sample1 and sample2.

Step 4: Interface implemented inside the class execute.

Step 5: In get1 method, get the name of the book.

Step 6: In get2 method, get the price, no of pages in the book and reading page per page.

Step 7: In display method, display the details of book with reading time for a book.

Step8: Stop the program.

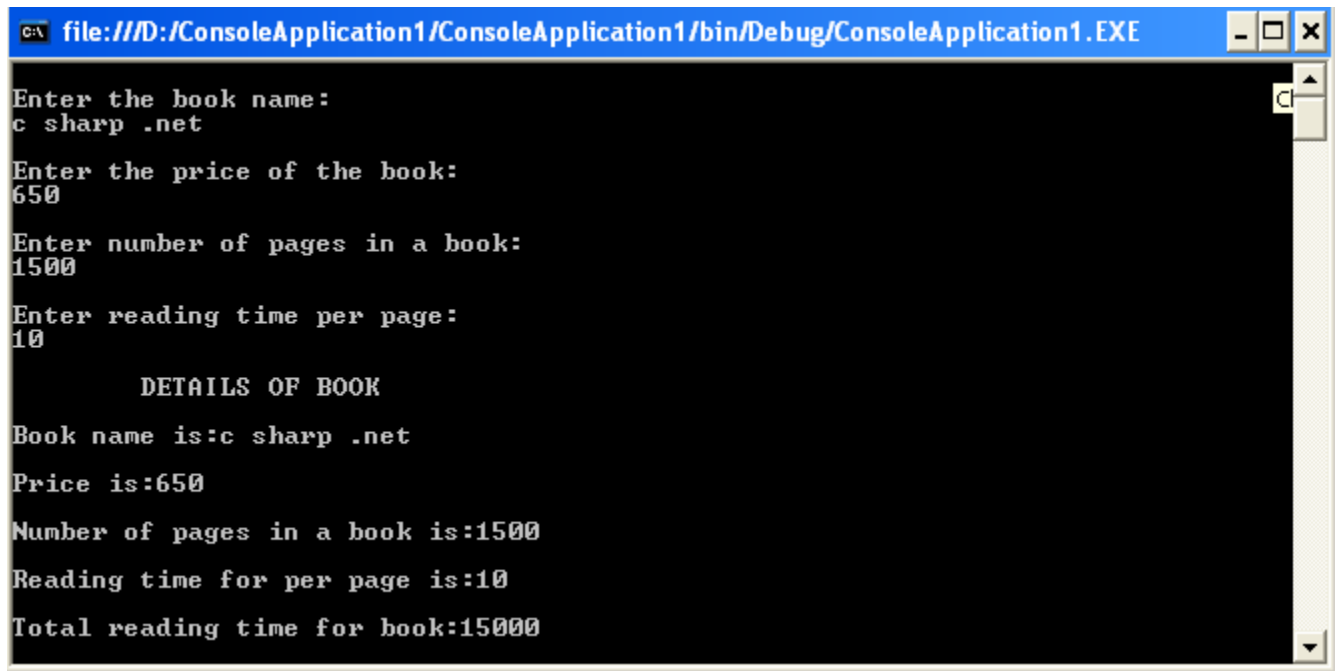
SOURCE CODE:

```
using System;
using System.Collections.Generic;
using System.Text;
namespace multiple
{
    public interface sample1
    {
        void get1();
    }
    public interface sample2
    {
        void get2();
    }
    class execute:sample1,sample2
    {
        int tot;
        int bpage;
        int rtime;
        string bname;
        int price;
        public void get1()
        {
            Console.WriteLine("\nEnter the book name:");
            bname = Console.ReadLine();
            Console.WriteLine("\nEnter the price of the book:");
            price = Convert.ToInt32(Console.ReadLine());
        }
        public void get2()
        {
            Console.WriteLine("\nEnter number of pages in a book:");
            bpage=Convert.ToInt32(Console.ReadLine());
            Console.WriteLine("\nEnter reading time per page:");
```



```
    rtime=Convert.ToInt32(Console.ReadLine());
}
public void display()
{
    Console.WriteLine("\n\tDETAILS OF BOOK");
    Console.WriteLine("\nBook name is:" + bname);
    Console.WriteLine("\nPrice is:" + price);
    Console.WriteLine("\nNumber of pages in a book is:" + bpage);
    Console.WriteLine("\nReading time for per page is:" + rtime);
    tot = bpage * rtime;
    Console.WriteLine("\nTotal reading time for book:" + tot);
}
public static void Main(string[] args)
{
    execute obj = new execute();
    obj.get1();
    obj.get2();
    obj.display();
    Console.ReadLine();
}
}
}
```

OUTPUT:



```
file:///D:/ConsoleApplication1/ConsoleApplication1/bin/Debug/ConsoleApplication1.EXE
Enter the book name:
c sharp .net
Enter the price of the book:
650
Enter number of pages in a book:
1500
Enter reading time per page:
10
      DETAILS OF BOOK
Book name is:c sharp .net
Price is:650
Number of pages in a book is:1500
Reading time for per page is:10
Total reading time for book:15000
```

RESULT:

Thus the program in C# to perform interface concept has been executed and the output is verified successfully.

Aim:

To write a C# program to implement the concept of binary operator overloading.

Algorithm:

- Step1. Start the program
- Step2. Construct a class named calculator.
- Step3. Define a constructor calculator()
 - a. number=0
- Step4. Define a constructor calculator(int)
 - a. number=n
- Step5. Overload a binary operator +(calculator o1, calculator o2)
 - a. Create an object of class calculator
 - b. Object.number=o1.number+o2.number
 - c. Return object
- Step6. Define a method display()
 - a. Print the number
- Step7. End the class calculator.
- Step8. Construct a class for main function named calnum
- Step9. Define the main()
- Step10. Create three objects for class calculator namely num1, num2, num3.
- Step11. Num3=num1+num2
- Step12. Num1.display()
- Step13. Num2.display()
- Step14. Num3.display()
- Step15. End the class calnum.
- Step16. Stop.

SOURCE CODE:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace overloadbinary
{
    class Calculator
    {
        public int number;
        public Calculator()
        {
            number=0;
        }
        public Calculator (int n)
        {
            number = n;
        }
        public static Calculator operator +(Calculator Calc1, Calculator Calc2)
        {
            Calculator Calc3 = new Calculator();
            Calc3.number= Calc2.number + Calc1.number;
            return Calc3;
        }
        public void display()
        {
            Console.WriteLine("{0}", number);
        }
    }
    class Calnum
    {
        static int Main(string[] args)
        {
            Calculator num1 = new Calculator (100);
            Calculator num2 = new Calculator (50);
            Calculator num3 = new Calculator ();
            num3 = num1 + num2;
            num1.display();
            num2.display();
            num3.display();
            Console.ReadLine();
            return 0; } } }
```

SAMPLE INPUT/OUTPUT:

A screenshot of a Windows console window. The title bar shows the file path: "file:///C:/Documents and Settings/DIGIT/Local Settings/Application Data/Temporary Projects/ConsoleAppli...". The console output displays three lines of text: "100", "50", and "150", each on a new line. The console has a black background and white text. The window includes standard Windows window controls (minimize, maximize, close) in the top right corner and a vertical scrollbar on the right side.

```
C:\> file:///C:/Documents and Settings/DIGIT/Local Settings/Application Data/Temporary Projects/ConsoleAppli...  
100  
50  
150
```

RESULT:

Thus the program in C# to perform binary operator overloading concept has been executed and the output is verified successfully.

AIM:

To write a C# program to implement the concept of unary operator overloading.

ALGORITHM:

- Step1. Start the program
- Step2. Construct a class calculator
- Step3. Define a constructor(int num1, int num2)
 - a. Number1= num1
 - b. Number2=num2
- Step4. Overload a unary operator -(calculator c1)
 - a. C1.number1=-c1.number1
 - b. C1.number2=-c1.number2
 - c. Return c1
- Step5. Define a method print()
 - a. Print number1 and number2
- Step6. End class calculator
- Step7. Construct a class for main function named entrypoint
- Step8. Define a method main()
 - a. Create an object for the class calculator by passing two numbers.
 - b. Calc=-calc
 - c. Calc.print()
- Step9. End the class entrypoint
- Step10. Stop the program.

SOURCE CODE:

```
using System;
namespace Calculator
{
    class Calculator
    {
        public int number1, number2;
        public Calculator(int num1, int num2)
        {
            number1 = num1;
            number2 = num2;
        }
        public static Calculator operator -(Calculator c1)
        {
            c1.number1 = -c1.number1;
            c1.number2 = -c1.number2;
            return c1;
        }
    }
}
```

```
    }  
    public void Print()  
    {  
        Console.WriteLine(" number1=" +number1);  
        Console.WriteLine(" number2=" +number2);  
        Console.ReadLine();  
    }  
}  
class EntryPoint  
{  
    static void Main()  
    {  
        Calculator calc = new Calculator(15,-25);  
        calc = -calc;  
        calc.Print();  
    }  
}  
}
```

SAMPLE INPUT/OUTPUT:

A screenshot of a Windows console window. The title bar at the top reads "C:\ file:///C:/Documents and Settings/DIGIT/Local Settings/Application Data/Temporary Projects/ConsoleAppli...". The console area is black with white text. The first line of text is "number1=-15" and the second line is "number2=25". There is a vertical scrollbar on the right side of the console window.

```
C:\ file:///C:/Documents and Settings/DIGIT/Local Settings/Application Data/Temporary Projects/ConsoleAppli...
number1=-15
number2=25
```

RESULT:

Thus the program in C# to perform unary operator overloading concept has been executed and the output is verified successfully.

AIM:

To write a C# program for implement delegate.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a class daycount with a function as same as class name.

Step 3: Declare month and year as zero using constructors.

Step 4: Using leapyear function, find the given year is leapyear or not.

Step 5: Using the switch case, give separate cases for each month.

Step 6: In display function, display the number of days in the concerned month.

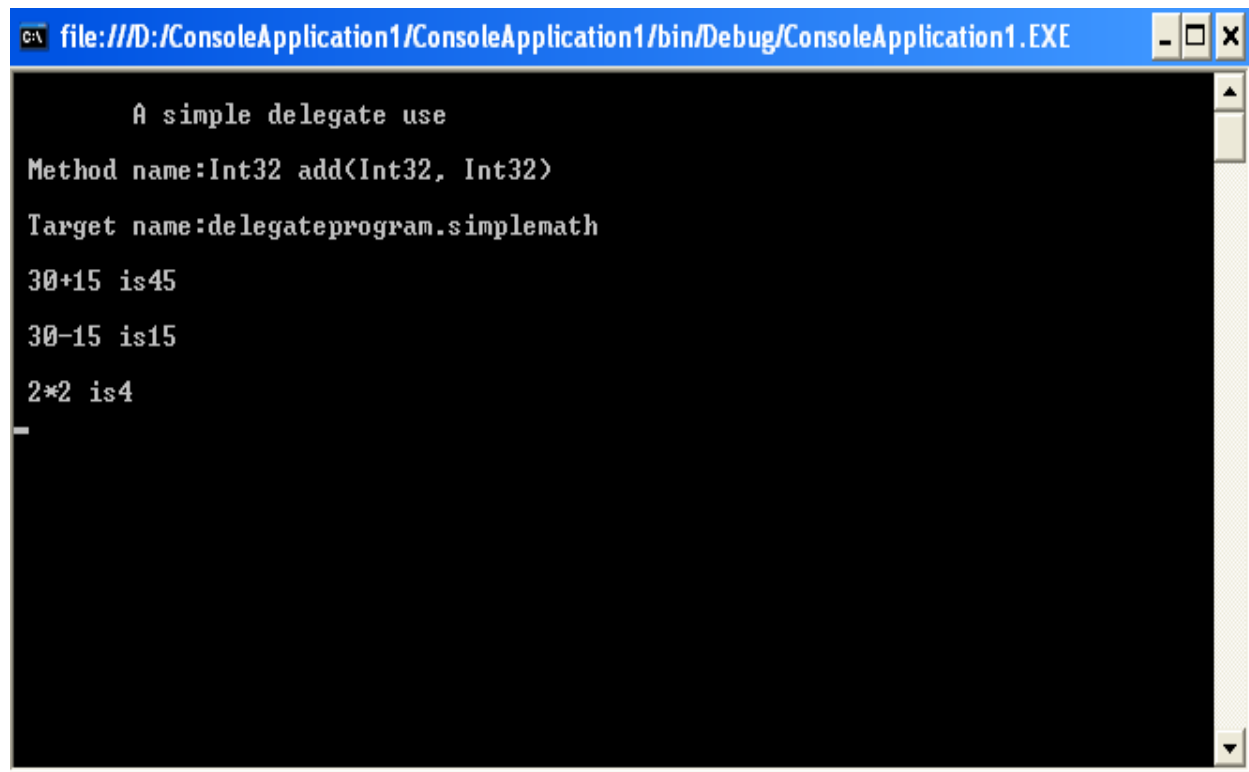
Step 7: Stop the program.

SOURCE CODE:

```
using System;
using System.Collections.Generic;
using System.Text;
namespace delegateprogram
{
    public delegate int binarymath(int x,int y);
    public delegate int squarenumber(int a);
    public class simplemath
    {
        public int add(int x,int y)
        {
            return x+y;
        }
        public int subtract(int x,int y)
        {
            return x-y;
        }
        public int square(int a)
        {
            return a*a;
        }
    }
    class simpledelegate
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("\n\tA simple delegate use");
            simplemath m = new simplemath();
            binarymath b = new binarymath(m.add);
            binarymath s = new binarymath(m.subtract);
            squarenumber x = new squarenumber(m.square);
            DisplayDelegateInfo(b);
            Console.WriteLine("\n 30+15 is{0} ", b(30,15));
        }
    }
}
```

```
Console.WriteLine("\n 30-15 is{0} ", s(30, 15));
Console.WriteLine("\n 2*2 is{0} ", x(2));
Console.ReadLine();
}
}
static void DisplayDelegateInfo(Delagate delobj)
{
foreach (Delegate d in delobj.GetInvocationList())
{
Console.WriteLine("\n Method name:{0}", d.Method);
Console.WriteLine("\n Target name:{0}", d.Target);
}
}
}
}
```

OUTPUT:



The screenshot shows a Windows console application window with a blue title bar. The title bar text is "file:///D:/ConsoleApplication1/ConsoleApplication1/bin/Debug/ConsoleApplication1.EXE". The console window has a black background with white text. The output text is as follows:

```
A simple delegate use
Method name: Int32 add(Int32, Int32)
Target name: delegateprogram.simplemath
30+15 is 45
30-15 is 15
2*2 is 4
```

RESULT:

Thus the C# program using delegate has been executed and the output is verified successfully.

AIM:

To write a program in C# to perform event concept.

ALGORITHM:

Step 1 : Start the program.

Step 2: declare a delegate with arguments.

Step 3: get the num value in thenumber variable using a method defined by delegate.

Step 4: create an event using the delegate.

Step5: in event, ShownOnScreen method is used to print the values to display the output.

Step 6: if $(i \% 7 == 0)$ is true, call the method ShownOnScreen to print the output and for loop will be incremented by one.

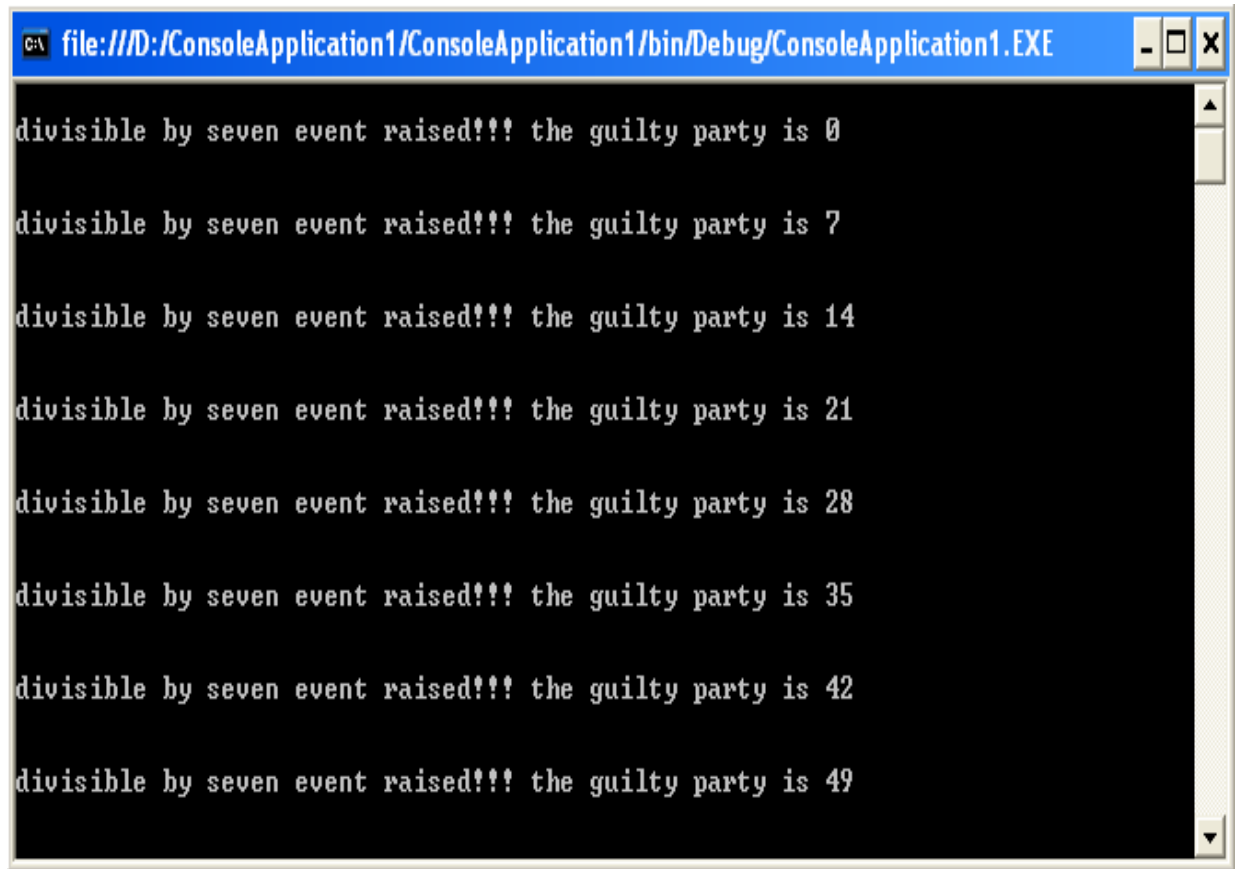
Step7: stop the program.

SOURCE CODE:

```
using System;
using System.Collections.Generic;
using System.Text;
namespace ConsoleApplication1
{
    public delegate void divbysevenhandler(object o,DivBySevenEventArgs e);
    public class DivBySevenEventArgs : EventArgs
    {
        public readonly int thenumber;
        public DivBySevenEventArgs (int num)
        {
            thenumber = num;
        }
    }
    public class divbysevenlistener
    {
        public void ShownOnScreen(object o,DivBySevenEventArgs e)
        {
            Console.WriteLine("\ndivisible by seven event raised!!! the guilty party is {0}",
            e.thenumber);
            Console.Read();
        }
    }
    public class busterboy
    {
        public static event divbysevenhandler eventseven;
        public static void Main(string[] args)
        {
            divbysevenlistener dbel = new divbysevenlistener();
            eventseven += new divbysevenhandler(dbel.ShownOnScreen);
            Gennumbers();
        }
        public static void OnEventSeven(DivBySevenEventArgs e)
        {
            if (eventseven != null)
            eventseven(new object(), e);
        }
        public static void Gennumbers()
        {
            for (int i = 0; i < 55; i++)
            {
```

```
if (i % 7 == 0)
{
    DivBySevenEventArgs el = new DivBySevenEventArgs(i);
    OnEventSeven(el);
    Console.Read();
}}
}
}
}
```

OUTPUT:



```
file:///D:/ConsoleApplication1/ConsoleApplication1/bin/Debug/ConsoleApplication1.EXE

divisible by seven event raised!!! the guilty party is 0

divisible by seven event raised!!! the guilty party is 7

divisible by seven event raised!!! the guilty party is 14

divisible by seven event raised!!! the guilty party is 21

divisible by seven event raised!!! the guilty party is 28

divisible by seven event raised!!! the guilty party is 35

divisible by seven event raised!!! the guilty party is 42

divisible by seven event raised!!! the guilty party is 49
```

RESULT:

Thus the program in C# to perform an event concept has been executed and the output is verified successfully.

AIM:

To write a C# sharp program to detect the exception occurrence.

ALGORITHM:

- | | |
|---------|--|
| Step 1 | Start the program |
| Step 2 | Create a class name program |
| Step 3 | Define a main function |
| Step 4 | Declare an array list of size 5. |
| Step 5 | Get the limit from the user n. |
| Step 6 | For(i=0;i<n;i++) |
| Step 7 | Get list[i] |
| Step 8 | If i+1==5 then print exception occurred. |
| Step 9 | End for |
| Step 10 | End the end the class. |
| Step 11 | Stop |

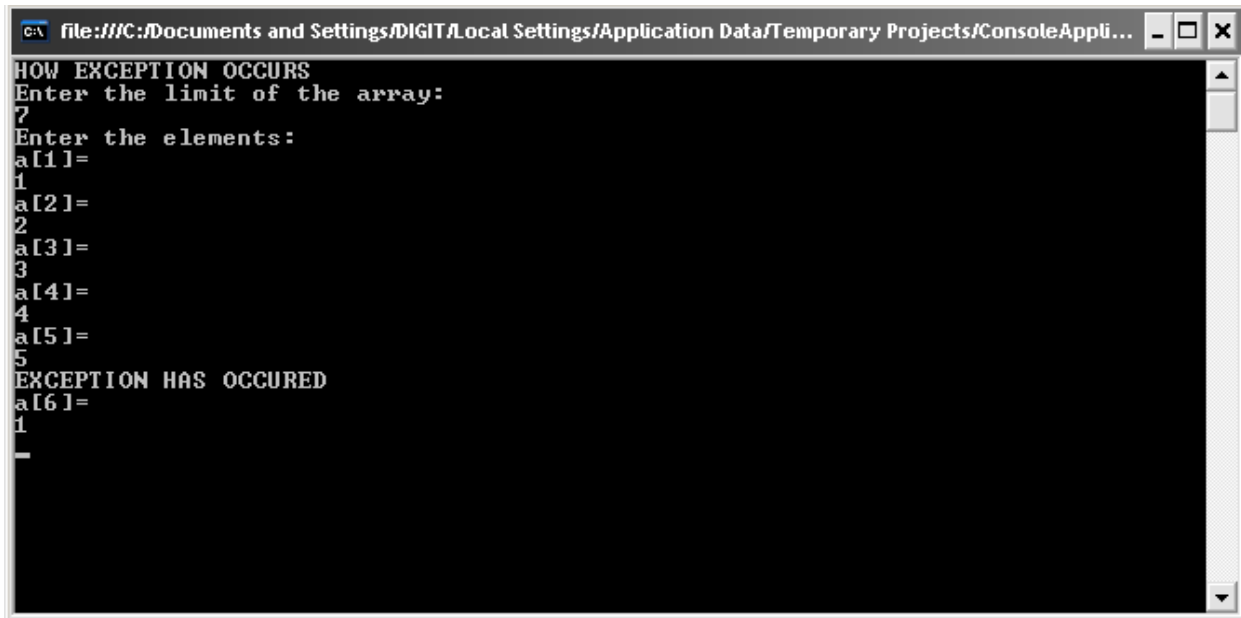
SOURCE CODE:

```
using System;
using System.Collections.Generic;
using System.Text;

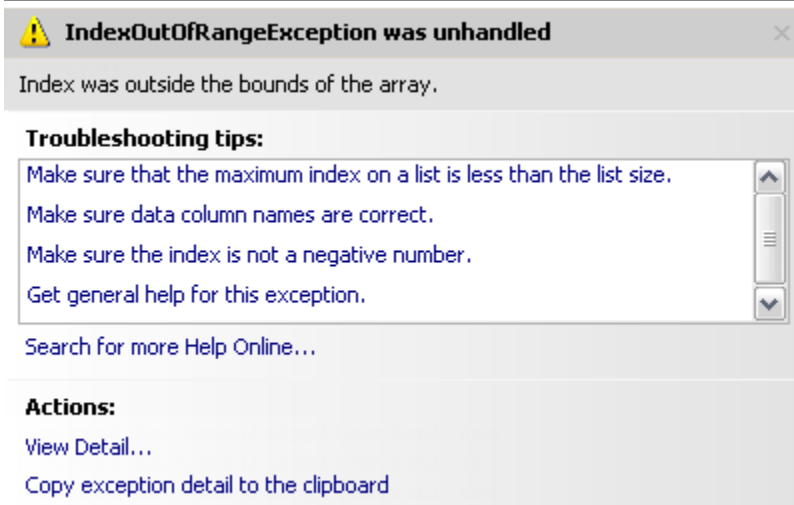
namespace exception
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] a=new int[5];
            int n;
            Console.WriteLine("HOW EXCEPTION OCCURS");
            Console.WriteLine("Enter the limit of the array:");
            n=int.Parse(Console.ReadLine());
            Console.WriteLine("Enter the elements:");
            for(int i=0;i<n;i++)
            {
                Console.WriteLine("a[{0}]=", i + 1);
                a[i]=int.Parse(Console.ReadLine());
                if(i+1==5)
                {
```

```
        Console.WriteLine("EXCEPTION HAS OCCURED");  
    }  
}  
Console.WriteLine("EXCEPTION");  
Console.ReadLine();  
}  
}
```

SAMPLE INPUT/OUTPUT:



```
file:///C:/Documents and Settings/DIGIT/Local Settings/Application Data/Temporary Projects/ConsoleAppli...
HOW EXCEPTION OCCURS
Enter the limit of the array:
7
Enter the elements:
a[1]=
1
a[2]=
2
a[3]=
3
a[4]=
4
a[5]=
5
EXCEPTION HAS OCCURED
a[6]=
1
_
```



RESULT:

Thus the program in C# to perform exception occurrence has been executed and the output is verified successfully.

AIM:

To write a C# program to implement the concept of exceptional handling.

ALGORITHM:

- Step 1 Start the program
- Step 2 Create a class named array.
- Step 3 Define the main method
 - i. Declare a array named list with size 5.
 - ii. Start try block
 - iii. Get the limit for the array as n
 - iv. For i=1 to n
 - v. Get list[i]
 - vi. Close try block
 - vii. Start catch block
 - viii. Print “index out of range”
 - ix. End catch block
- Step 4 end the method
- Step 5 end the class
- Step 6 stop.

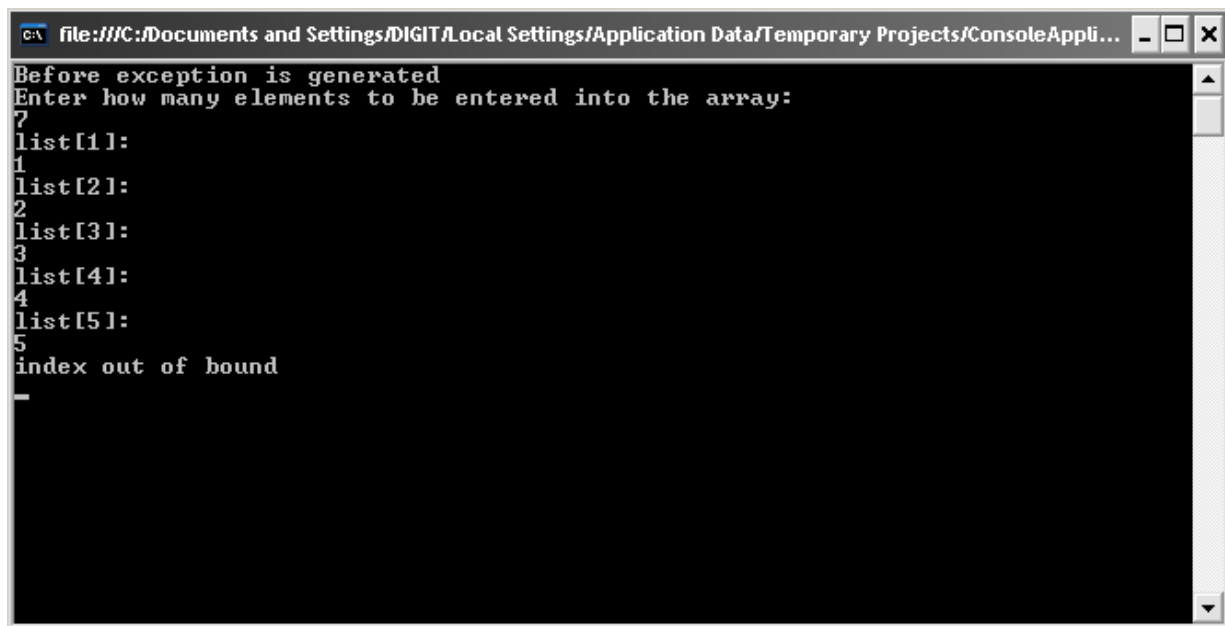
SOURCE CODE:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace exception
{
    class Array
    {
        static void Main(string[] args)
        {
            int[] list = new int[5];
            int n;
            try
            {
                Console.WriteLine("Before exception is generated");
                Console.WriteLine("Enter how many elements to be entered into the array:");
```

```
n = int.Parse(Console.ReadLine());
for (int i = 1; i < n; i++)
{
    Console.WriteLine("list[{0}]:", i);
    list[i] = int.Parse(Console.ReadLine());
}
}
catch (IndexOutOfRangeException)
{
    Console.WriteLine("index out of bound");
    Console.ReadLine();
}
}
}
```

SAMPLE INPUT/OUTPUT:



```
file:///C:/Documents and Settings/DIGIT/Local Settings/Application Data/Temporary Projects/ConsoleAppli...
Before exception is generated
Enter how many elements to be entered into the array:
7
list[1]:
1
list[2]:
2
list[3]:
3
list[4]:
4
list[5]:
5
index out of bound
_
```

RESULT:

Thus the program in C# to perform Exception handling concept has been executed and the output is verified successfully.

AIM:

To write a C# program to implement the concept of multithreading.

ALGORITHM:

Step1.Start.

Step2.Create a class server class for threading option .

Step3.create a method void Instance() .

Step4. Enter the sleep time for the Thread A

Step5.using void Readline Method throw exception if it occurs .

Step6.create a method static () and run thread B and run the sleep time.

Step7.In main class create objects for the methods and execute various threads.

Step8.stop.

SOURCE CODE:

```
using System;
using System.Threading;
public class serverclass
{
    public void InstanceMethod()
    {
        Console.WriteLine("You are in instance Method running on thread A");
        Console.WriteLine("Thread A going to sleep");
        Thread.Sleep(10000);
        Console.WriteLine("you are in Instance Method. Running on thread A");
        Console.ReadLine();
    }

    private void readline()
    {
        throw new Exception("The method or operation is not implemented.");
    }

    public static void staticMethod()
    {
        Console.WriteLine("You are in start Method .Running on thread B");
        Console.WriteLine("Thread B going to sleep");
        Thread .Sleep(10000);
        Console.WriteLine("you are back in static method .running on thread B");
        Console.ReadLine();
    }
}
public class simple
{
    public static int Main(String[]args)
```

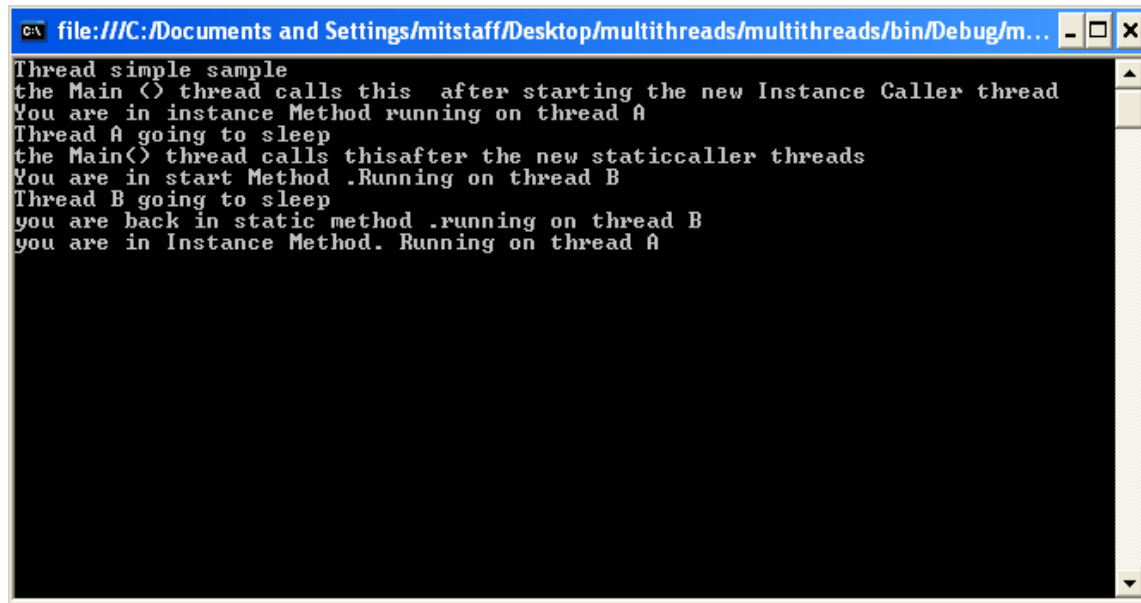
```

{
    Console.WriteLine("Thread simple sample");
    serverclass serverobject = new serverclass();
    Thread InstanceCaller=new Thread (serverobject.InstanceMethod);
    InstanceCaller.Start();
    Console.WriteLine("the Main () thread calls this "+" after starting the new Instance Caller
thread");
    Thread Staticcaller=new Thread (new ThreadStart(serverclass.staticMethod));
    Staticcaller.Start();
    Console.WriteLine("the Main() thread calls this"+"after the new staticcaller threads");
    return 0;
}

private static void readline()
{
    throw new Exception("The method or operation is not implemented.");
}
}

```


OUTPUT:



```
file:///C:/Documents and Settings/mitstaff/Desktop/multithreads/multithreads/bin/Debug/m...
Thread simple sample
the Main (>) thread calls this after starting the new Instance Caller thread
You are in instance Method running on thread A
Thread A going to sleep
the Main(>) thread calls thisafter the new staticcaller threads
You are in start Method .Running on thread B
Thread B going to sleep
you are back in static method .running on thread B
you are in Instance Method. Running on thread A
```

RESULT:

Thus the program in C# to perform Multithreading concept has been executed and the output is verified successfully

AIM:

To write a C# program to implement the concept of ADO.NET.

ALGORITHM:

Step1 : Start

Step2 :Add a form,drag the list box for displaying the data entered in the database.

Step3:Drag for text boxes for emp no,name,salary&dept.

Step4:Drag the buttons to perform the required operations.

Step5:In show button ,display the selected item in list box

Step 6:In add button , insert the new item in to the database.

Step 7:Using update button alter the data of the item selected.

Step8:Delete button used to remove the selected item.

Step9:Use clear button to remove the entries in the textbox.

Step10:Stop.

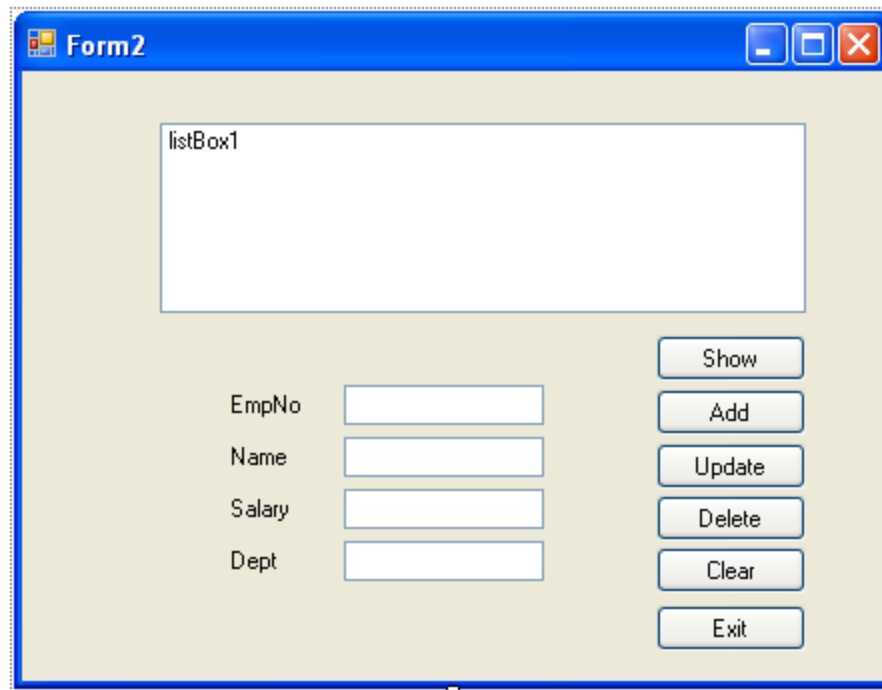
SOURCE CODE:**PROGRAM .CS:**

```
using System;
using System.Collections.Generic;
using System.Windows.Forms;

namespace MECDemo
{
    static class Program
    {
        /// <summary>
        /// The main entry point for the application.
        /// </summary>
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Form2());
            ///Application.Run(new Form1());
        }
    }
}
```

```
}  
}
```

FORM2(DESIGN):



FORM2:

```
using System;  
using System.Collections.Generic;  
using System.ComponentModel;  
using System.Data;  
using System.Data.SqlClient;  
using System.Drawing;  
using System.Text;  
using System.Windows.Forms;
```

```
namespace MECDemo
```

```
{
```

```
    public partial class Form2 : Form  
    {
```

```

public Form2()
{
    InitializeComponent();
}
private SqlConnection mySqlConnection;
private SqlDataAdapter mySqlDataAdapter;
private DataSet myDataSet;
private DataTable myDataTable = new DataTable();
private void Form2_Load(object sender, EventArgs e)
{
    LoadData();
}

private void LoadData()
{
    try
    {
        String conStr = "server=MITLAB2-
SYS24\\SQLEXPRESS;database=test;uid=sa;pwd=12345";
        mySqlConnection = new SqlConnection(conStr);
        mySqlConnection.Open();
        mySqlDataAdapter = new SqlDataAdapter("SELECT * FROM Employee",
mySqlConnection);
        myDataSet = new DataSet("Employee");
        mySqlDataAdapter.Fill(myDataSet, "Employee");
        listBox1.DataSource = myDataSet.Tables[0];
        listBox1.DisplayMember = "EName";
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message.ToString());
    }
}

private void button4_Click(object sender, EventArgs e)
{
    this.Close();
}

```

```

private void ClearData()
{
    LoadData();

    txtEmpNo.Text = "";
    txtName.Text = "";
    txtSalary.Text = "";
    txtDept.Text = "";
}

private void button1_Click(object sender, EventArgs e)
{
    //Add Data
    SqlCommand mySqlCommand = new SqlCommand("INSERT INTO Employee
(EName,Salary,Dept) VALUES (@ENAME,@Salary,@Dept)", mySqlConnection);
    mySqlCommand.Parameters.AddWithValue("@ENAME", txtName.Text);
    mySqlCommand.Parameters.AddWithValue("@Salary", txtSalary.Text);
    mySqlCommand.Parameters.AddWithValue("@Dept", txtDept.Text);
    mySqlCommand.ExecuteNonQuery();
    ClearData();
}

private void button3_Click(object sender, EventArgs e)
{
    // Delete
    string empno = txtEmpNo.Text;

    SqlCommand mySqlCommand = new SqlCommand("DELETE FROM Employee
WHERE Eid = @Eid", mySqlConnection);
    mySqlCommand.Parameters.AddWithValue("@Eid", txtEmpNo.Text);
    mySqlCommand.ExecuteNonQuery();
    ClearData();
}

private void button2_Click(object sender, EventArgs e)
{

```

```

        // Update Data
        SqlCommand mySqlCommand = new SqlCommand("UPDATE Employee SET EName
= @EName, Salary = @Salary, Dept = @Dept WHERE Eid = @Eid", mySqlConnection);
        mySqlCommand.Parameters.AddWithValue("@Eid", txtEmpNo.Text);
        mySqlCommand.Parameters.AddWithValue("@EName", txtName.Text);
        mySqlCommand.Parameters.AddWithValue("@Salary", txtSalary.Text);
        mySqlCommand.Parameters.AddWithValue("@Dept", txtDept.Text);
        mySqlCommand.ExecuteNonQuery();
        ClearData();
    }

    private void button5_Click(object sender, EventArgs e)
    {
        ClearData();
    }

    private void button6_Click(object sender, EventArgs e)
    {
        // Show Data
        string Eid = txtEmpNo.Text;

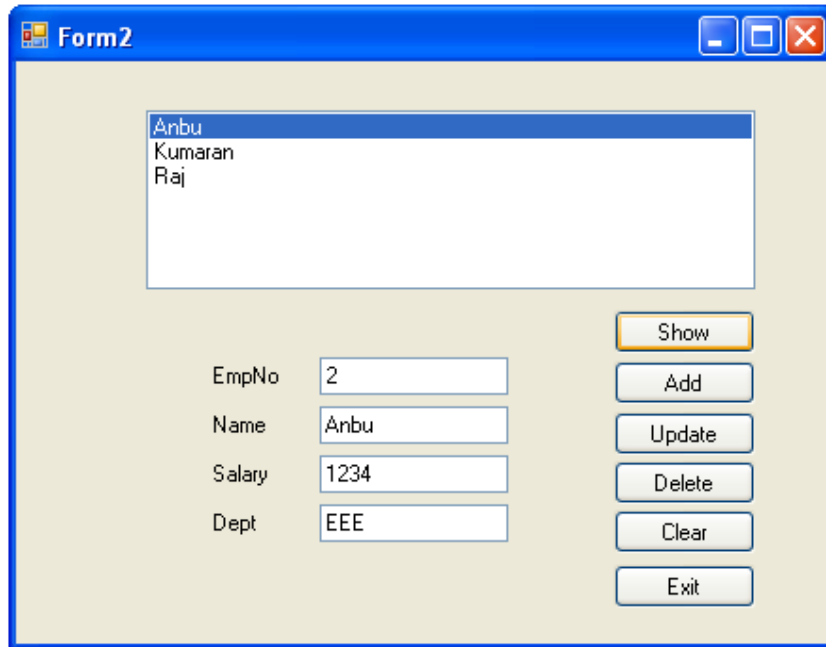
        SqlCommand mySqlCommand = new SqlCommand("SELECT * FROM Employee
WHERE Eid = @Eid", mySqlConnection);
        mySqlCommand.Parameters.AddWithValue("@Eid", Eid);

        SqlDataReader mySQLDataReader = mySqlCommand.ExecuteReader();
        if (mySQLDataReader.Read())
        {
            txtName.Text = mySQLDataReader["EName"].ToString();
            txtSalary.Text = mySQLDataReader["Salary"].ToString();
            txtDept.Text = mySQLDataReader["Dept"].ToString();
        }
        mySQLDataReader.Close();
        mySqlCommand.Dispose();
    }

    private void listBox1_SelectedIndexChanged(object sender, EventArgs e)
    { } } }

```

OUTPUT:



Database name : Text

Table name : Employee

TABLE CREATION :

Field	Datatype	Size
Eid	int	(1,1) NOT NULL,
EName	varchar	(100) NULL,
Salary	int	NULL,
Dept	varchar	100NULL

RESULT:

Thus the program in C# to perform ADO.NET concept has been executed and the output is verified successfully.

AIM:

To write a VB.NET program for implementing the concept of functional overloading in polymorphism.

ALGORITHM:

- Step 1 Start the program.
- Step 2 Create a class calculatemax
- Step 3 Define a function max with two parameters of integer type and return type as integer.
 - a. If no1>no2 return no1.
 - b. Else return no2.
- Step 4 Define another function of same constituents but with the parameter of single type.
 - a. If no1>no2 return no1.
 - b. Else return no2.
- Step 5 End the class.
- Step 6 Create another class maxcalc
- Step 7 Define main method
 - a. Create an object for the class calculatemax as calc
 - b. Calc(5.4F, 8.6F)
 - c. Calc(19,12)
- Step 8 End the class.
- Step 9 Stop.

SOURCE CODE:

```
Imports System.Collections.Generic
Imports System.Text
Namespace CalculateMax
    Class CalculateMax
        Public Function Max(number1 As Integer, number2 As Integer) As Integer
            If number1 > number2 Then
                Return number1
            Else
                Return number2
            End If
        End Function
        Public Function Max(number1 As Single, number2 As Single) As Single
            If number1 > number2 Then
                Return number1
            Else
                Return number2
            End If
        End Function
    End Class
    Class MaxCalc
        Private Shared Function Main(args As String()) As Integer
            Dim calc As New CalculateMax()
            System.Console.WriteLine("{0}", calc.Max(5.4F, 8.6F))
            System.Console.WriteLine("{0}", calc.Max(19, 12))
            System.Console.ReadLine()
            Return 0
        End Function
    End Class
End Namespace

Module Module1

    Sub Main()
        Dim calc As New CalculateMax.CalculateMax()
        System.Console.WriteLine("{0}", calc.Max(5.4F, 8.6F))
        System.Console.WriteLine("{0}", calc.Max(19, 12))
        System.Console.ReadLine()
    End Sub

End Module
```

SAMPLE INPUT/OUTPUT:

A screenshot of a Windows console window. The title bar shows the file path: "file:///C:/Documents and Settings/DIGIT/Local Settings/Application Data/Temporary Projects/ConsoleAppli...". The console area is black with white text. The first line of output is "8.6" and the second line is "19". The window has standard Windows controls (minimize, maximize, close) in the top right corner.

RESULT:

Thus the program in VB.NET to perform polymorphism concept has been executed and the output is verified successfully.

AIM:

To write a vb.net program to implement the concept of inheritance.

ALGORITHM:

- Step1. Start the program
- Step2. Create a namespace programcall
- Step3. Construct a base class named Base
 - Step3.1. Create a method named GetAB()
 - Step3.1.1. Get the values of a and b.
 - Step3.2. Create a method named printAB()
 - Step3.2.1. print the values of a and b.
- Step4. End of base class.
- Step5. Create a derived class named Derived : Base
 - Step 5.1. Create a method named Get()
 - Step5.1.1. call GetAB()
 - Step5.1.2. Get value for c.
 - Step5.2. Create a method named Print()
 - Step5.2.1. call printAB()
 - Step5.2.2. print the value of c.
- Step6. End of derived class.
- Step7. Construct a class for the main fuction named mainclass
 - Step7.1. Create a method named main()
 - Step7.2. Create an object for the Derived class.
 - Step7.3. call the methods Get(), Print(), printAB() using the object.
- Step8. End the main class.
- Step9. End the program.

SOURCE CODE:

Module Module1

Class Base

Private A As Integer, B As Integer

Public Sub GetAB()

Console.Write("Enter Value For A : ")

A = Integer.Parse(Console.ReadLine())

Console.Write("Enter Value For B : ")

B = Integer.Parse(Console.ReadLine())

End Sub

Public Sub PrintAB()

Console.Write("A = {0}" & vbTab & "B = {1}" & vbTab, A, B)

End Sub

End Class

Class Derived

Inherits Base

Private C As Integer

Public Sub [Get]()

'Accessing base class method in derived class

GetAB()

Console.Write("Enter Value For C : ")

C = Integer.Parse(Console.ReadLine())

End Sub

Public Sub Print()

'Accessing base class method in derived class

PrintAB()

Console.WriteLine("C = {0}", C)

End Sub

End Class

Sub Main()

Dim obj As New Derived()

obj.[Get]()

obj.Print()

'Accessing base class method with derived class instance

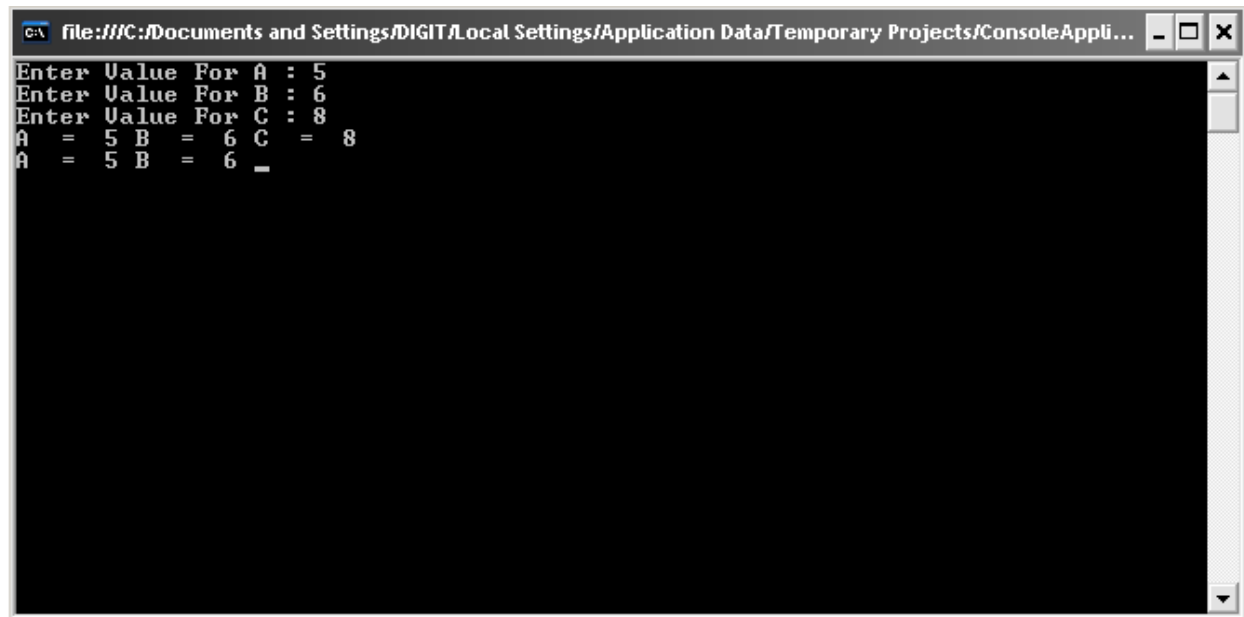
obj.PrintAB()

Console.Read()

End Sub

End Module

SAMPLE INPUT/OUTPUT:



A screenshot of a Windows console application window. The title bar shows the file path: "file:///C:/Documents and Settings/DIGIT/Local Settings/Application Data/Temporary Projects/ConsoleAppli...". The window contains the following text:

```
Enter Value For A : 5
Enter Value For B : 6
Enter Value For C : 8
A = 5 B = 6 C = 8
A = 5 B = 6 _
```

RESULT:

Thus the program in VB.NET to perform inheritance concept has been executed and the output is verified successfully.

AIM:

To write a VB.net program using windows application to check whether the given alphabet is vowel or not vowel.

ALGORITHM:

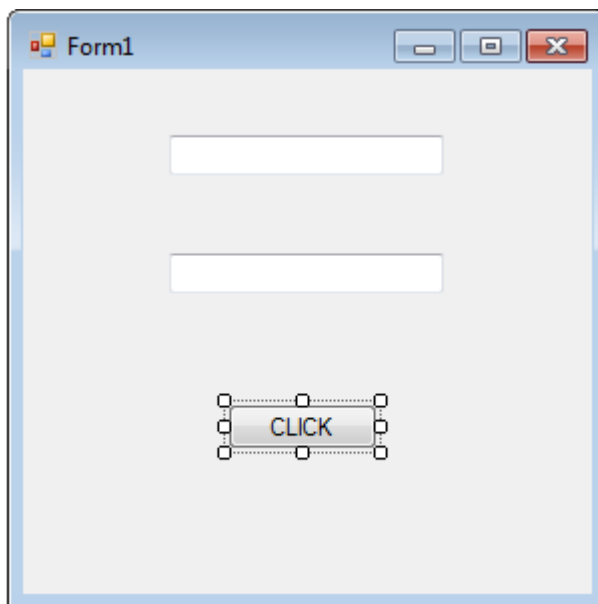
Step 1: Start the program.

Step 2: Drag two textboxes and a button from the tool box.

Step 3: Type a letter in textbox1.

Step 4: If you click the button, it will show vowel or not vowel in the textbox2.

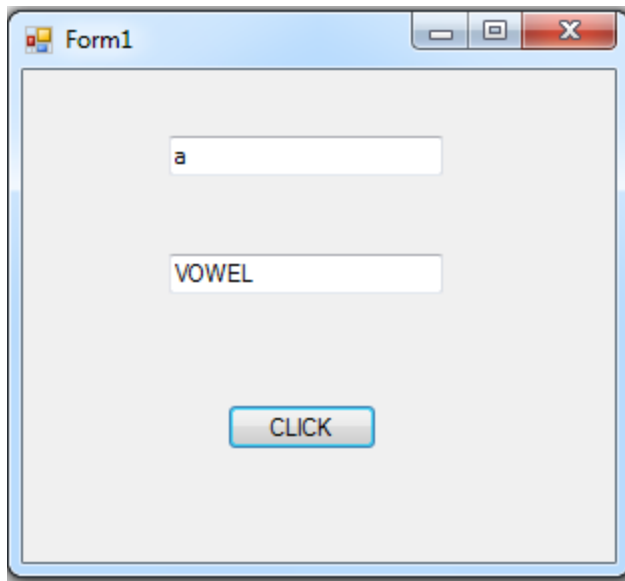
Step 5: Stop the program.

WINDOWS FORM:

PROGRAM:

```
Public Class Form1
Private Sub Button2_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles Button2.Click
Dim C As Char
C = UCase(TextBox1.Text)
If C = "A" Or C = "E" Or C = "I" Or C = "O" Or C = "U" Then
TextBox2.Text = "VOWEL"
Else
TextBox2.Text = "NOT VOWEL"
End If
End Sub
End Class
```

OUTPUT:



RESULT:

Thus the VB.net program using windows application has been executed and the output is verified successfully.

AIM:

To write a VB.net program to implement the concept of layout managers and containers.

ALGORITHM:

- Step 1 Start
- Step 2 Create a form1 and drag the tab control from container block.
- Step 3 Create the necessary boxes to perform addition of two no.
- Step 4 Use enable, remove and add blocks to add and remove the tabpages.
- Step 5 Create a form2 and drag the flow layout and table layout.
- Step 6 Drag the buttons in to the layout created.
- Step 7 Create a panel in the same form2.
- Step 8 Create a form3 and perform the operation to contrast the tree layout.
- Step 9 Stop.

SOURCE CODE:

Form design:

Public Class Form1

```
Private Sub CheckBox1_CheckedChanged(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles CheckBox1.CheckedChanged
```

```
    If CheckBox1.Checked = True Then
```

```
        TabPageName.Enabled = True
```

```
    ElseIf CheckBox1.Checked = False Then
```

```
        TabPageName.Enabled = False
```

```
    End If
```

End Sub

```
Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles Button1.Click
    TabPageName.Controls.Remove(TabPage1)
End Sub
```

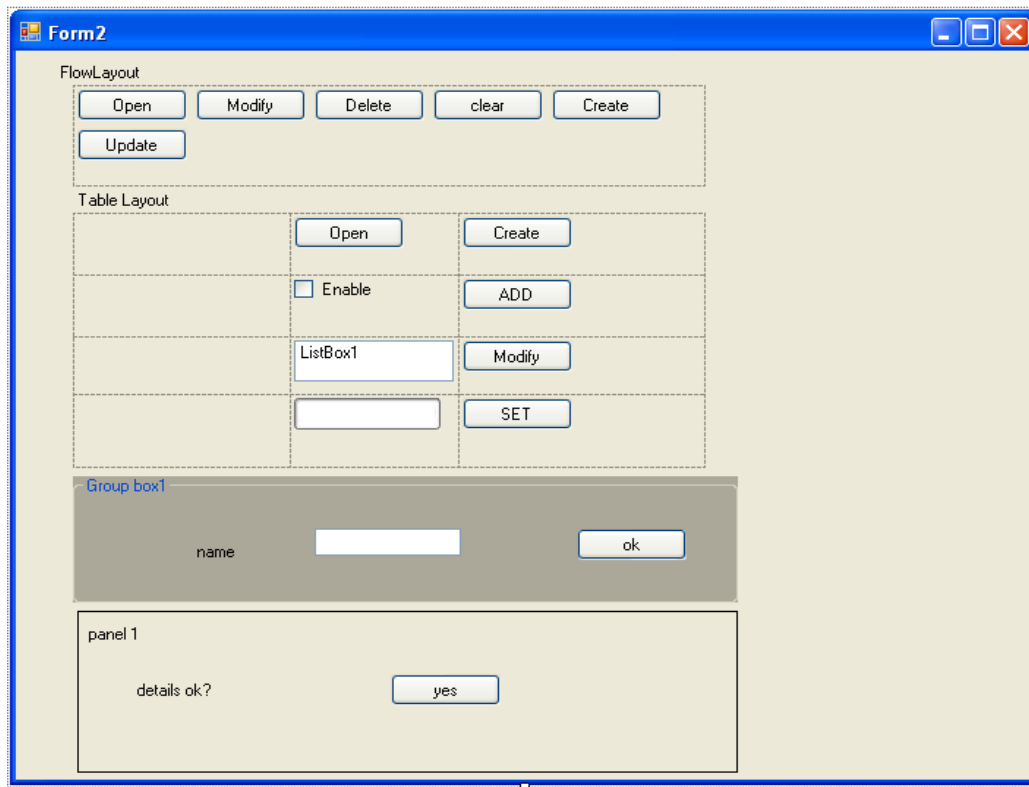
```
Private Sub Button2_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles Button2.Click
    Dim tabPage4 As New TabPage
    tabPage4.Text = "Tab Page 4"
    TabPageName.TabPages.Add(tabPage4)
End Sub
```

```
Private Sub Button3_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles Button3.Click
    'MessageBox.Show(CInt(TextBox1.Text) + CInt(TextBox2.Text).ToString(), "MIT")

    TextBox3.Text = CInt(TextBox1.Text) + CInt(TextBox2.Text).ToString()
```

```
End Sub
End Class
```

Form2 Design



Form2

FlowLayout

Open Modify Delete clear Create

Update

Table Layout

	Open	Create
	☐ Enable	ADD
ListBox1		Modify
		SET

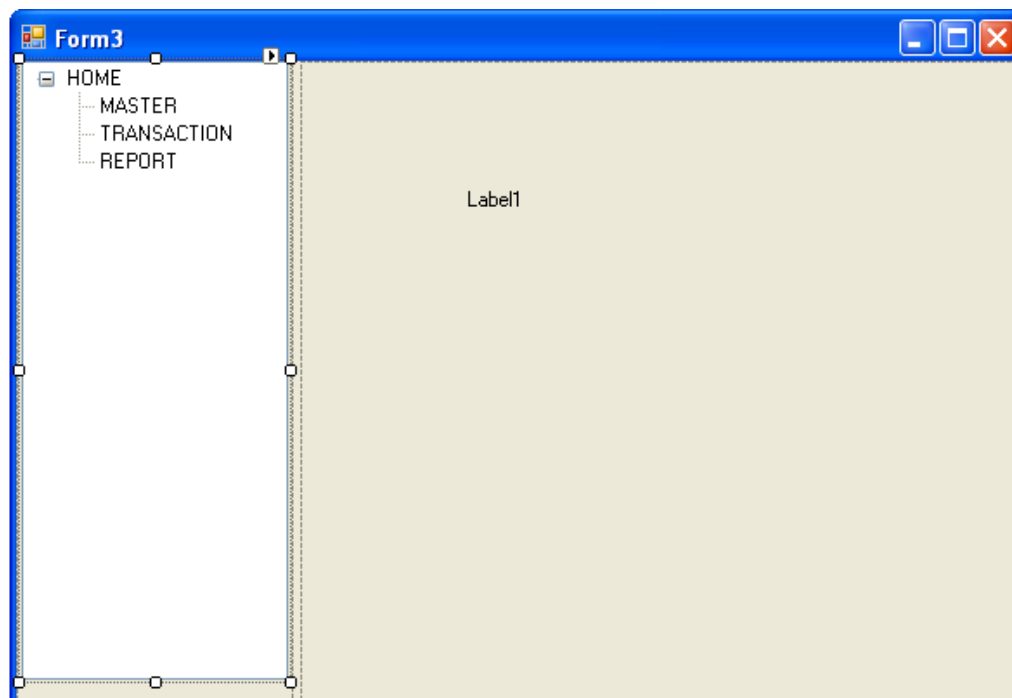
Group box1

name ok

panel 1

details ok?

Form3 design:



Form3

HOME

- MASTER
- TRANSACTION
- REPORT

Label1

```
Public Class Form3
```

```
    Private Sub TreeView1_AfterSelect(ByVal sender As System.Object, ByVal e As  
System.Windows.Forms.TreeViewEventArgs) Handles TreeView1.AfterSelect
```

```
        Label1.Text = TreeView1.SelectedNode.ToString()
```

```
    End Sub
```

```
End Class
```

OUTPUT:

Internet - Form1 - 172.16.11.14

TabPage1 TabPage2 Tab Page 4

1
15
16

Add

☐ Enable Remove ADD

Form2

FlowLayout

Open Modify Delete clear Create

Update

Table Layout

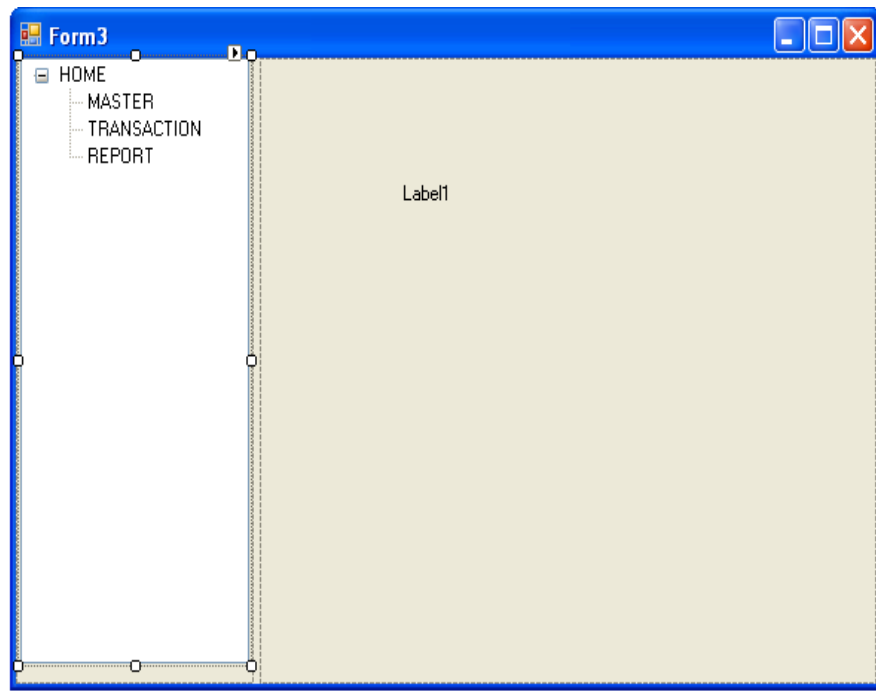
	Open	Create
	☐ Enable	ADD
	ListBox1	Modify
		SET

Group box1

name ok

panel 1

details ok?



RESULT:

Thus the VB.net program using layout manager and containers has been executed and the output is verified successfully.

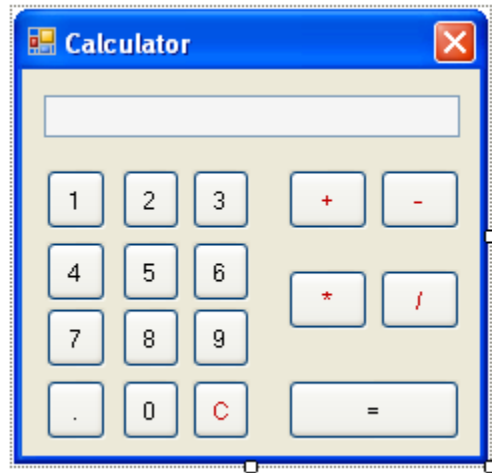
AIM:

To write a VB.net program in single document interface (SDI) forms.

ALGORITHM:

- Step1. Start the program.
- Step2. Create a class named frmcalculator.
- Step3. Declare two operands 1 & 2 of double type.
- Step4. Declare a variable [operator] of string type.
- Step5. Define a method to handle button 1 to 9 and button 0.
- Step6. Get the button no. in the textbox.
- Step7. Define a method to handle clear button
- Step8. Clear the textbox.
- Step9. Define methods to handle add, sub, mul, div as follows.
- Step10. Operand1 = value in text box.
- Step11. Clear textbox
- Step12. [operator]=(+or-or*/or/) as per the method.
- Step13. Define a method to handle decimal button.
- Step14. If InStr(txtsource.Text, ".") > 0 Then exit
- Step15. Else txtsource.Text = txtsource.Text & "."
- Step16. Define a method to handle equal button.
- Step17. Result = value in textbox.
- Step18. Use switch statement for [operator]
- Step19. Case "+"
- Step20. Result= operand1+operand2
- Step21. Case "-"
- Step22. Result= operand1-operand2
- Step23. Case "*"
- Step24. Result= operand1*operand2
- Step25. Case "/"
- Step26. Result= operand1/operand2
- Step27. End the class
- Step28. Stop.

SOURCE CODE:



```
Public Class frmcalculator
    Dim Operand1 As Double
    Dim Operand2 As Double
    Dim [Operator] As String
```

```
    Private Sub btn1_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
        Handles btn1.Click, btn2.Click, btn3.Click, btn4.Click, btn5.Click, btn6.Click, btn7.Click,
        btn8.Click, btn9.Click, btnzero.Click
        txtsource.Text = txtsource.Text & sender.text
    End Sub
```

```
    Private Sub btnclear_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
        Handles btnclear.Click
        txtsource.Text = ""
    End Sub
```

```
    Private Sub btnadd_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
        Handles btnadd.Click
        Operand1 = Val(txtsource.Text)
        txtsource.Text = ""
        txtsource.Focus()
        [Operator] = "+"
    End Sub
```

```

End Sub
Private Sub btndecimal_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles btndecimal.Click
    If InStr(txtsource.Text, ".") > 0 Then
        Exit Sub
    Else
        txtsource.Text = txtsource.Text & "."
    End If
End Sub

Private Sub btnequals_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles btnequals.Click
    Dim Result As Double
    Operand2 = Val(txtsource.Text)

    Select Case [Operator]
        Case "+"
            Result = Operand1 + Operand2
            MsgBox(Result.ToString("#,###.00"), MsgBoxStyle.Information, "Result")
            txtsource.Text = Result.ToString("#,###.00")
        Case "-"
            Result = Operand1 - Operand2
            MsgBox(Result.ToString("#,###.00"), MsgBoxStyle.Information, "Result")
            txtsource.Text = Result.ToString("#,###.00")
        Case "/"
            Result = Operand1 / Operand2
            MsgBox(Result.ToString("#,###.00"), MsgBoxStyle.Information, "Result")
            txtsource.Text = Result.ToString("#,###.00")
        Case "*"
            Result = Operand1 * Operand2
            MsgBox(Result.ToString("#,###.00"), MsgBoxStyle.Information, "Result")
            txtsource.Text = Result.ToString("#,###.00")
    End Select

    txtsource.Text = Result.ToString("#,###.00")

End Sub

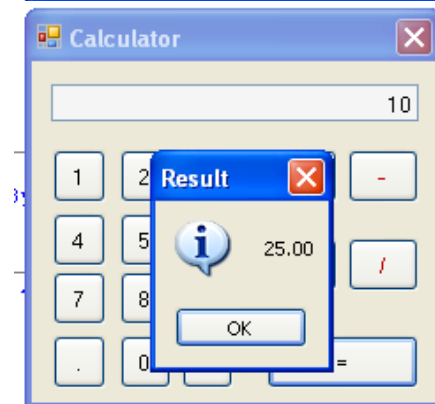
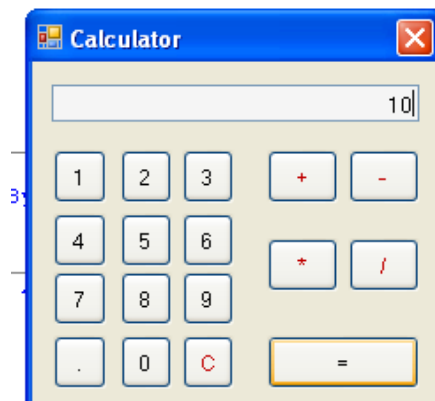
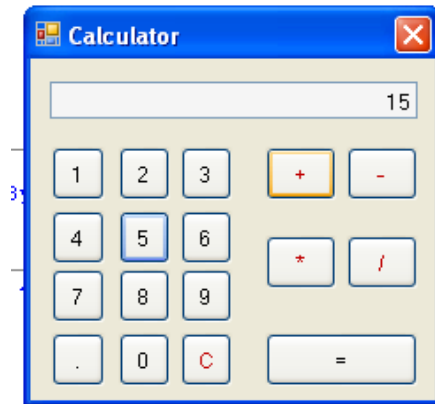
```

```
Private Sub btnminus_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles btnminus.Click
    Operand1 = Val(txtsource.Text)
    txtsource.Text = ""
    txtsource.Focus()
    [Operator] = "-"
End Sub
```

```
Private Sub btnmultiply_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles btnmultiply.Click
    Operand1 = Val(txtsource.Text)
    txtsource.Text = ""
    txtsource.Focus()
    [Operator] = "*"
End Sub
```

```
Private Sub btndivide_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles btndivide.Click
    Operand1 = Val(txtsource.Text)
    txtsource.Text = ""
    txtsource.Focus()
    [Operator] = "/"
End Sub
End Class
```

SAMPLE INPUT/OUTPUT:



RESULT:

Thus the VB.net program using Single document interface forms has been executed and the output is verified successfully.

AIM:

To write a VB.net program in multiple document interface (MDI) forms.

ALGORITHM:

- Step 1 Start
- Step 2 Create a MDI form from the solution explorer by adding a class to a form
- Step 3 Create two forms in the file block of the MDI form.
- Step 4 Add necessary blocks to process students mark list in form1.
- Step 5 In form1 get the student marks and calculate the total.
- Step 6 Add necessary blocks to process students details in form2.
- Step 7 In form2 get student name ,reg no,mark and calculate the average and status.
- Step 8 Access both forms by executing MDI.
- Step 9 Stop.

SOURCE CODE:

FORM1 DESIGN

Public Class Form1

Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles Button1.Click

TextBox6.Text = CInt(TextBox1.Text) + CInt(TextBox2.Text) + CInt(TextBox3.Text) +
CInt(TextBox4.Text) + CInt(TextBox5.Text).ToString()

End Sub
End Class

The screenshot shows a Windows application window titled "Form1". The window has a blue title bar with minimize, maximize, and close buttons. The main area has a beige background. At the top, the text "STUDENTS MARK LIST" is displayed in a green, italicized, serif font. Below this, there are six labels: "TAMIL", "ENGLISH", "MATHS", "SCIENCE", "SOCIAL", and "TOTAL", each followed by a white text box. At the bottom center, there is a button labeled "calculate".

FORM2 DESIGN:

Form2

STUDENT DETAILS

NAME

REGISTER NUMBER

TOTAL

AVERAGE

STATUS

RESULT

Public Class Form2

Private Sub TextBox4_TextChanged(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles TextBox4.TextChanged

End Sub

Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles Button1.Click

TextBox4.Text = CInt(TextBox3.Text / 5).ToString()

If (TextBox4.Text >= 50) Then

TextBox5.Text = "pass"

Else

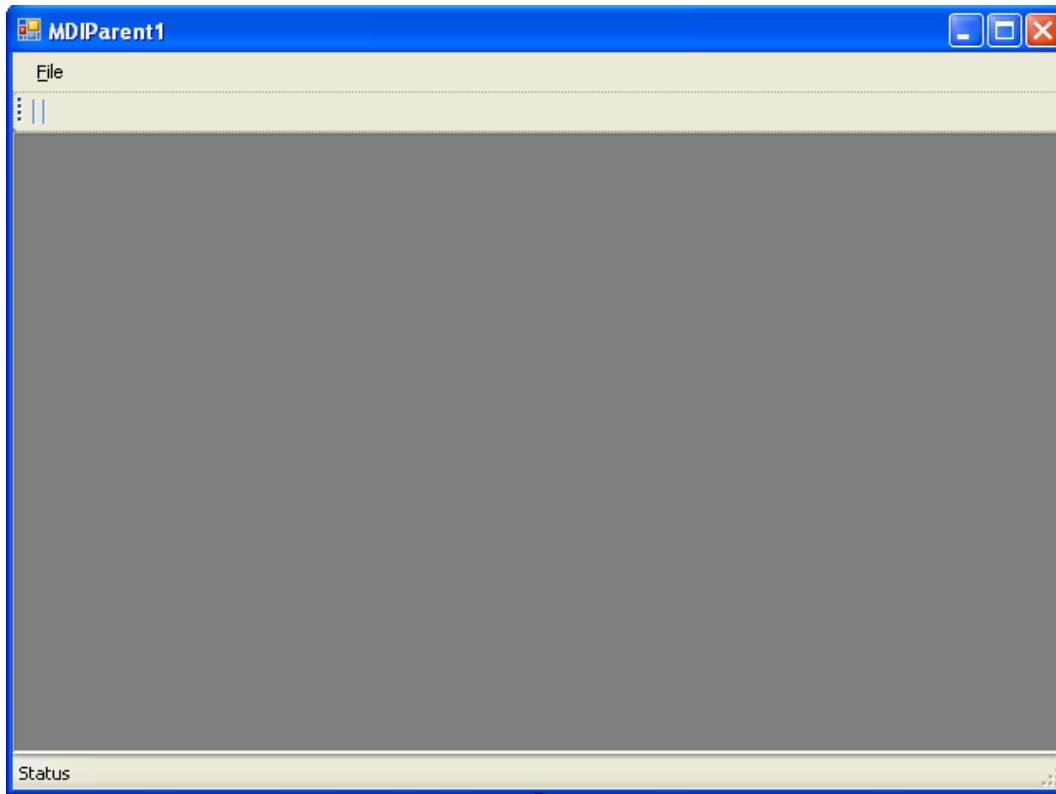
TextBox5.Text = "fail"

End If

End Sub

End Class

MDI FORM DESIGN:



```
Imports System.Windows.Forms
```

```
Public Class MDIParent1
```

```
    Private m_ChildFormNumber As Integer = 0
```

```
    Private Sub MARKLISTToolStripMenuItem_Click(ByVal sender As System.Object, ByVal e  
As System.EventArgs) Handles MARKLISTToolStripMenuItem.Click
```

```
        Dim f As New Form1()
```

```
        f.Show()
```

```
    End Sub
```

```
    Private Sub DETAILToolStripMenuItem_Click(ByVal sender As System.Object, ByVal e  
As System.EventArgs) Handles DETAILToolStripMenuItem.Click
```

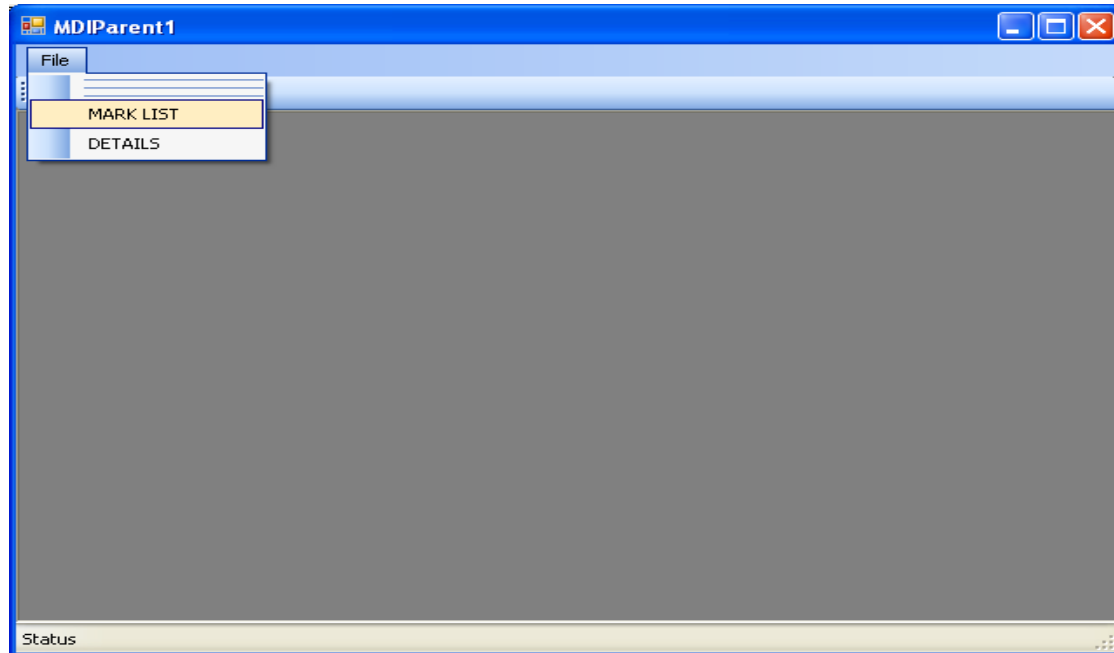
```
        Dim f As New Form2()
```

```
        f.Show()
```

```
    End Sub
```

```
End Class
```


OUTPUT:



The screenshot shows the same application window with the "MARK LIST" option selected. The main area now displays a form titled "STUDENTS MARK LIST" in green, italicized text. The form contains several input fields with labels to their left:

- TAMIL: 95
- ENGLISH: 96
- MATHS: 100
- SCIENCE: 91
- SOCIAL: 94
- TOTAL: 476

Below these fields is a button labeled "calculate".

The screenshot shows a Windows form titled "Form2" with a blue title bar. The form has a light beige background and displays the following information:

- STUDENT DETAILS** (in a stylized, italicized green font)
- NAME**: Text box containing "RAJA"
- REGISTER NUMBER**: Text box containing "111"
- TOTAL**: Text box containing "476"
- AVERAGE**: Text box containing "95"
- STATUS**: Text box containing "pass"
- RESULT**: A button with a dashed border and the text "RESULT" in red.

RESULT:

Thus the VB.net program using Single document interface forms has been executed and the output is verified successfully.